htblakaindorf
Leistung mit Menschlichkeit

INFORMATIK

Kontrollstrukturen

POS1-Q2, Kontrollstrukturen

Erstellungsdatum: Aug. 2017

Autoren:
DR. JOHANNES FARMER
DI CHRISTOPH KOHLWEG
DI URSULA RIESEL
DR. HEINZ SCHIFFERMÜLLER
DI FLORIAN SCHREIBER
DI JOHANN WURZINGER

Überarbeitung: Juli 2019 / RI

Kompetenzen

Algorithmen
- Dezimal-Binär-Konverter, 17
- Dreieckstyp, 5
- Eingabestatistik, 17
- Fakultät I, 12
- Fakultät II, 22
- Fibonacci Reihe, 19
- größter gemeinsamer Teiler, 18
- k-te Quadratwurzel nach Heron, 21
- Kreislinie, 6
- Lottozahlen II, 23
- Minimum / Maximum, 6
- Primfaktorenzerlegung, 18
- Quadratwurzel nach Heron, 21
- Weihnachtsbaum, 24
- Weihnachtsbaumverkäufer-App, 23
- Zahlen raten II, 22
- Zahlen sortieren I, 5
- Zahlen sortieren II, 5
- Zahlenanalyse, 7
- Zahlensumme, 6
- Ziffernsumme III, 17

Operatoren
- Bitmanipulation I, 16
- Bitmanipulation II, 16
- Dreieckstyp, 5
- Kreislinie, 6
- Minimum / Maximum, 6
- Quadrantensuche I, 6
- Stellenrechner, 8
- Taschenrechner, 8
- Testergebnis, 7
- Textanalyse, 7
- Zahlen sortieren II, 5
- Zahlenanalyse, 7
- Zahlensumme, 6

Struktogramme
- größter gemeinsamer Teiler, 18
- Unbekannt I, 16

Kontrollstrukturen
- Armstrong Zahlen, 14
- Bestimmung der Jahreszeit, 9
- Bitmanipulation I, 16
- Bitmanipulation II, 16
- Dezimal-Binär-Konverter, 17
- Dreieckstyp, 5
- Durchschnitt I, 11
- Eingabestatistik, 17
- Fakultät I, 12
- Fakultät II, 22
- Fibonacci Reihe, 19
- Freier Fall, 13
- größter gemeinsamer Teiler, 18
- Hexadezimale Zahl, 10
- k-te Quadratwurzel nach Heron, 21
- Kalender, 9
- Kapitalrechner, 21
- Kleine if-Beispiele, 4
- Kreislinie, 6
- Lottozahlen I, 12
- Lottozahlen II, 23
- Minimum / Maximum, 6
- Mittelwertberechnung I, 18
- Mittelwertberechnung II, 19
- Näherungsverfahren für PI, 15
- Nummernausgabe I, 10
- Nummernausgabe II, 10
- Plausibilitätspruefung I, 19
- Plausibilitätspruefung II, 20
- Potenzieren, 13
- Primfaktorenzerlegung, 18
- Primzahl, 13
- Pythagoräisches Tripel, 23
- Quadrantensuche I, 6
- Quadrantensuche II, 14
- Quadrantensuche III, 15
- Quadratwurzel nach Heron, 21
- Quadratzahlen, 11
- Römische Zahlen, 9
- Rechteckflaeche, 21
- Stellenrechner, 8
- Summe I, 11
- Summe II, 11
- Summenberechnung, 20
- Taschenrechner, 8
- Teiler finden, 14
- Temperaturtabelle, 12
- Testergebnis, 7
- Textanalyse, 7
- Turmrechnung, 14
- Unbekannt I, 16
- Ungerade Zahlen, 11
- Würfeln, 12
- Würfelspiel I, 19

Würfelspiel II, 19
Weihnachtsbaum, 24
Weihnachtsbaumverkäufer-App, 23
Wochentagsbestimmung, 9
Zahlen raten I, 22
Zahlen raten II, 22
Zahlen sortieren I, 5
Zahlen sortieren II, 5
Zahlenanalyse, 7
Zahlenspiel, 8
Zahlensumme, 6
Zahlentester, 23
Zeichen analysieren, 17
Zeichen zählen, 16
Ziffernsumme I, 13
Ziffernsumme II, 13
Ziffernsumme III, 17
zufälliger Großbuchstabe, 20
Zufallszahlen, 20

Coding-Conventions

Folgende Codierungsrichtlinien sind beim Programmieren strikt einzuhalten:

- Naming-Conventions:
  - Syntaxregeln für Bezeichner: erlaubte Buchstaben, Ziffern und Sonderzeichen
    keine Umlaute, kein 'ß'
  - Für alle Bezeichner sind sprechende, **englische** Namen zu verwenden.
    Ausnahme: für Laufvariable dürfen einzelne Buchstaben verwendet werden
  - CamelCase: Großbuchstaben innerhalb eines Wortes (z.B.: **maxValue**, **indexOf(), ...**)
  - Kleinschreibug für alle anderen Bezeichner (Variablen, Fuktionsnamen, ...)
  - nur Großbuchstaben für symbolische Konstanten (z.B.: **MAX, DIM, ...**
- Indentation:
  - **Einrücken!!!**
  - Zeilenumbrüche
  - Zeilenlänge max. 80 Zeichen
  - Autoformatter hot keys: `Ctrl + K + D`
- Leerzeichen:
  - zwischen Operatoren: a = b + c;
  - eine Leerzeile zwischen zwei Funktionen, nach dem `include`-Block bzw. dem `define`-Block sowie nach einem Variablendeklarationsblock
- Kommentare:
  - Kommentare für alle selbst implementierten Funktionen
  - Implementierungs-Kommentare
- nur eine Anweisung pro Zeile
- nur eine Deklaration pro Zeile

Q2-1 Aufgaben zur `if`-Anweisung

Q2-101 Kleine if-Beispiele ⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein C-Programm **LittleIfExamples**, das folgende kurze if-Beispiele hintereinander ausführt:

- Eine ganze Zahl wird eingelesen. Falls die Zahl positiv oder Null ist, wird die Wurzel berechnet, andernfalls wird eine entsprechende Fehlermeldung ausgegeben.

  ```
  Bitte eine ganze Zahl eingeben: 4
  Die Wurzel aus 4 betraegt 2.

  Bitte eine ganze Zahl eingeben: -3
  Die Wurzel aus einer negativen Zahl ist nicht moeglich.
  ```

- Es sind drei Gleitkommazufallszahlen zu erzeugen. Es soll festgestellt werden, ob die Summe der drei Zahlen größer ist als ihr Produkt oder umgekehrt.

  ```
  1. Zufallszahl: -1.0
  2. Zufallszahl: 2.0
  3. Zufallszahl: 1.5
  Die Summe 2.50 ist groesser als das Produkt -3.00.
  ```

- Es ist eine ganzzahlige Zufallszahl aus dem Intervall [-99..+99] zu erzeugen. Es soll ausgegeben werden, wie viele Stellen die Zufallszahl hat.
  Folgende Ausgaben sind möglich:

  ```
  Die Zufallszahl lautet 9 und ist einstellig!
  Die Zufallszahl lautet -89 und ist zweistellig!
  ```

- Überprüfe, ob eine eingegebene ganze Zahl gerade und ein Vielfaches von 3 ist.
  Folgende Ausgaben sind möglich:

  ```
  Die Zahl 12 ist gerade und ein Vielfaches von 3
  Die Zahl 4 erfuellt nicht die geforderten Bedingungen.
  ```

  Zusatz: Gib im nein Zweig aus, welche Bedingung (ungerade oder kein Vielfaches von 3) nicht erfüllt ist.

- Von einem eingegebenen Zeichen ist zu bestimmen, ob es sich um einen Groß- oder Kleinbuchstaben handelt.
  Folgende Ausgaben sind möglich.

  ```
  Das Zeichen 'G' ist ein Großbuchstabe.
  Das Zeichen 'e' ist ein Kleinbuchstabe.
  Das Zeichen ',' ist kein Buchstabe.
  ```

  Zusatz: Überprüfe, ob es sich bei einem Buchstaben um einen Vokal handelt.

- Von einer eingegebenen 4-stelligen Jahreszahl ist zu bestimmen, ob es sich um ein Schaltjahr handelt.
  Hinweis: Ein Jahr ist ein Schaltjahr, wenn die Jahreszahl durch 4 teilbar ist. Weiters gilt: Vielfache von 100 sind keine Schaltjahre, Vielfache von 400 sind Schaltjahre.
  Folgende Ausgabe wird auf der Konsole erzeugt:

  ```
  Das Jahr 2000 ist ein Schaltjahr.
  ```

  oder:

  ```
  Das Jahr 2100 ist kein Schaltjahr.
  ```

Q2-102 Zahlen sortieren I ⋆

Kompetenzen: Kontrollstrukturen, Algorithmen

Erstelle ein C-Programm **NumberSort1**, das zwei ganze Zahlen einliest und sie in den Variablen `number1` und `number2` speichert. Die beiden Zahlen werden anschließend aufsteigend sortiert. Dabei werden die Variablenwerte vertauscht, wenn der Wert von `number1` größer ist als der von `number2`. Die sortierten Zahlen werden im folgenden Format auf der Konsole ausgegeben:

```
Zahl 1: 321
Zahl 2: 123
Eingegebene Zahlen: 321, 123
Zahlen sortiert:    123, 321
```

Q2-103 Zahlen sortieren II ⋆⋆

Kompetenzen: Kontrollstrukturen, Operatoren, Algorithmen

Erstelle ein C-Programm **NumberSort2**, das drei ganze Zahlen einliest und auf die Variablen `number1`, `number2` und `number3` speichert. Diese werden anschließend aufsteigend sortiert. Dabei werden die Variablenwerte vertauscht, bis gilt: `number1` < `number2` < `number3`. Auf der Konsole werden die Zahlen vor und nach dem Sortieren ausgegeben:

```
Zahl 1: 8
Zahl 2: 1
Zahl 3: 4
Eingegebene Zahlenfolge: 8, 1, 4
Sortierte Zahlenfolge:   1, 4, 8
```

Q2-104 Dreieckstyp ⋆⋆

Kompetenzen: Kontrollstrukturen, Operatoren, Algorithmen

Erstelle ein C-Programm **TypeOfTriangle**, das die Längen der Seiten a, b und c eines Dreiecks einliest.
Zuerst ist zu prüfen, ob c die längste Seite ist, andernfalls sind die Seiten entsprechend zu vertauschen. Dann soll der Typ des Dreiecks bestimmt werden:

- gleichseitig ($a = b = c$)
- gleichschenkelig ($a = b \neq c$ oder $a \neq b = c$ oder $a = c \neq b$)
- rechtwinkelig ($a^2 + b^2 = c^2$)
- allgemein ($a + b > c$)
- kein Dreieck (sonst)

Der Typ des Dreiecks ist auszugeben.

```
=====================================
Dreiecksbestimmung:
=====================================
Laenge Seite a: 3
Laenge Seite b: 4
Laenge Seite c: 5
Art des Dreiecks: rechtwinkelig
=====================================
```

oder

```
=====================================
Dreiecksbestimmung:
=====================================
Laenge Seite a: 3
Laenge Seite b: 4
Laenge Seite c: 3
```

```
Art des Dreiecks: gleichschenkelig
=====================================
```

Q2-105 Minimum / Maximum ⋆⋆

Kompetenzen: Kontrollstrukturen, Operatoren, Algorithmen

Erstelle ein C-Programm **MinMax**, das sechs Zufallszahlen im Bereich von 10 bis 50 (inklusive) erzeugt und das Minimum bzw. das Maximum aus diesen sechs Zufallszahlen ermittelt und ausgibt.
Folgende Ausgabe soll auf der Konsole erzeugt werden:

```
****************************************
MinMax-Bestimmung:
****************************************
Zufallszahlen: 34 - 17 - 41 - 23 - 27 - 32
Groesste Zahl: 41
Kleinste Zahl: 17
****************************************
```

Q2-106 Zahlensumme ⋆

Kompetenzen: Kontrollstrukturen, Operatoren, Algorithmen

Erstelle ein C-Programm **SumOfNumbers**, das 5 Zufallszahlen im Bereich von -60 bis +60 (inklusive) erstellt. Anschließend werden alle positiven und alle negativen Zahlen getrennt summiert und diese beiden Summenwerte ausgegeben.
Es wird folgende Ausgabe auf der Konsole erzeugt:

```
Zahlensumme:
-------------------------------------------
Zufallszahlen: 24/-16/53/4/-49
Summe der positiven Zahlen: 81
Summe der negativen Zahlen: -65
-------------------------------------------
```

Q2-107 Quadrantensuche I ⋆⋆

Kompetenzen: Kontrollstrukturen, Operatoren

Erstelle ein C-Programm **Quadrants1**, das die Koordinaten eines Punktes einliest und ermittelt, in welchem Quadranten (I-IV) sich der Punkt befindet.
Folgende Ausgabe wird auf der Konsole erzeugt:

```
Eingabe der x-Koordinate des Punktes: -3
Eingabe der y-Koordinate des Punktes: 6

Der Punkt (-3/6) liegt im 2. Quadranten.
```

Q2-108 Kreislinie ⋆⋆⋆

Kompetenzen: Kontrollstrukturen, Operatoren, Algorithmen

Erstelle ein C-Programm **CircleLine**, das für einen Kreis und einen Punkt berechnet, ob sich der Punkt innerhalb, außerhalb oder auf der Kreislinie befindet. Der Kreis wird dabei durch seine Mittelpunktkoordinaten und seinen Radius definiert. Der Punkt P wird durch seine Koordinaten definiert. Diese fünf Werte werden von der Konsole eingelesen.
Berechnungshinweis: Berechne den Abstand zwischen Kreismittelpunkt und P. Ermittle die Lage von P durch Vergleichen des Abstandes mit dem Radius.

```
Eingabe der x-Koordinate des Punktes: 4
Eingabe der y-Koordinate des Punktes: 12
Eingabe der x-Koordinate des Mittelpunktes: 3
Eingabe der y-Koordinate des Mittelpunktes: 3
Eingabe der Radiuslaenge: 5

Der Punkt (4/12) liegt ausserhalb der Kreislinie.
```

Q2-109 Zahlenanalyse ⋆⋆

Kompetenzen: Kontrollstrukturen, Operatoren, Algorithmen

Erstelle ein C-Programm **NumberAnalysis**, das eine ganze Zahl einliest (maximal 5-stellig) und sie anschließend analysiert:

- Ist die Zahl positiv ($\geq$0) oder negativ (<0) ?
- Ist die Zahl gerade oder ungerade?
- Wie viele Stellen hat die Zahl?

Folgende Ausgabe wird auf der Konsole erzeugt:

```
*******************************
Eingabe der Zahl: -9463
*******************************
Analyse der Zahl -9463 :
negativ - ungerade - 4-stellig
*******************************
```

Q2-110 Testergebnis ⋆⋆

Kompetenzen: Kontrollstrukturen, Operatoren

Erstelle ein C-Programm **TestResults**, das aus einer Punktezahl von einem Testergebnis eine Beurteilung in Form einer Note erzeugt. Es sind maximal 48 Punkte zu erreichen. Bei fehlerhafter Eingabe, d.h. Punkte < 0 oder Punkte > 48, bricht das Programm mit einer Fehlermeldung ab. Bei korrekter Eingabe der Punkte wird die Note anhand des folgenden Schlüssels berechnet:
45 - 48 Sehr gut
39 - 44 Gut
32 - 38 Befriedigend
25 - 31 Genügend
0 - 24 Nicht genügend

```
erreichte Punkte: 40
40 Punkte entsprechen der Note Gut.
```

```
erreichte Punkte: 1
1 Punkt entspricht der Note Nicht genuegend.
```

Q2-111 Textanalyse ⋆⋆

Kompetenzen: Kontrollstrukturen, Operatoren

Erstelle ein C-Programm **TextAnalysis**, mit dem die ersten 6 Zeichen eines Textes analysiert werden. Dabei werden die Anzahl an Großbuchstaben, Kleinbuchstaben, Ziffern und Sonderzeichen gezählt. Weiters wird die Summe aller ASCII-Werte berechnet.

```
Texteingabe: +M25+?
Textanalyse:
    Grossbuchstaben: 1
    Kleinbuchstaben: 0
```

```
Ziffern: 2
Sonderzeichen: 3
ASCII-Summe: 329
```

Q2-112 Stellenrechner ⋆⋆

Kompetenzen: Kontrollstrukturen, Operatoren

Erstelle ein C-Programm **NumberOfDigits**, das aus einer ganzen, positiven oder negativen Zahl mit maximal 6 Stellen berechnet, wie viele Stellen die Zahl hat. Hat die Zahl mehr als 6 Stellen so wird eine Fehlermeldung ausgegeben.

```
Bitte eine Zahl eingeben: 54321
Die Zahl 54321 hat 5 Stellen

Bitte eine Zahl eingeben: 1234567
Die Zahl hat zu viele Stellen!
```

Q2-113 Taschenrechner ⋆

Kompetenzen: Kontrollstrukturen, Operatoren

Erstelle ein C-Programm **Calculator**, das eine ganze Zahl, einen Operator(+,-,*,/) und eine zweite ganze Zahl einliest. Anschließend wird die entsprechende Berechnung ausgeführt und das Ergebnis mit der Rechnung auf der Konsole ausgegeben. (Achtung: Eine Division durch NULL darf nicht zugelassen werden!)

```
1. Zahl eingeben: 4
Rechenoperator eingeben (+, -, *, /): +
2. Zahl eingeben: 5
Rechnung: 4 + 5 = 9

1. Zahl eingeben: 4
Rechenoperator eingeben (+, -, *, /): /
2. Zahl eingeben: 0
Rechnung: Division durch Null nicht erlaubt!
```

Q2-2 Aufgaben zur `switch`-Anweisung

Q2-201 Zahlenspiel ⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein C-Programm **NumberGame**, das eine beliebige ganze Zahl einliest und je nach Rest bei Division durch 5 folgende Aktionen durchführt:

- Ist die Zahl durch 5 teilbar, dann soll die Zahl verdoppelt werden.
- Beträgt der Rest 2, dann soll die Zahl halbiert werden.
- Beträgt der Rest 3, dann soll die Zahl um 3 vergrößert werden.
- In allen übrigen Fällen ist die Zahl um 4 zu vermindern.

```
Bitte eine ganze Zahl eingeben: 17322
neuer Zahlenwert: 8661
```

Q2-202 Wochentagsbestimmung ⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein C-Programm **WeekDay**, das zu einem Datum den entsprechenden Wochentag ermittelt und ausgibt.

Hinweis:
Im Jahre 1582 wurde der gregorianische Kalender eingeführt. Mit Hilfe der Formel von Christian Zeller lässt sich zu jedem Datum ab dem Jahr 1582 der zugehörige Wochentag w ermitteln, wobei dem Sonntag die 1, dem Montag die 2 usw. entspricht:

w = (t + ((m + 1) * 26 / 10) + j + j / 4 + h / 4 - 2 * h) % 7
mit t = Tag, m = Monat, h = Hunderterzahl der Jahreszahl, j = Jahr innerhalb des Jahrhunderts.

Alle Divisionen sind ganzzahlig durchzuführen.
Für einige Tage kann es Probleme geben, etwa wenn der Dividend negativ wird, d.h. wenn w<0 gilt). Dann muss man den Wert 7 zu w addieren.

Beispiel: 6. 7. 1964. Hier gilt t = 6, m = 7, j = 64, h = 19 w = 72%7 = 2
Also war der 6. Juli 1964 ein Montag.

```
Bitte ein Datum eingeben: 6.7.1964
Der 6.Juli 1964 ist ein Montag.
```

Q2-203 Kalender ⋆⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein C-Programm **Calendar**, das aus einem numerisch eingegebenen Datum folgende Informationen ermittelt:

- um den wievielten Tag des Jahres es sich handelt
- der Name des Monats als Text

Es wird angenommen, dass der Monat Februar 28 Tage hat!

```
Bitte den Tag eingeben: 3
Bitte den Monat eingeben: 2

Der 3.Februar ist der 34.Tag des Jahres.
```

Q2-204 Bestimmung der Jahreszeit ⋆⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein C-Programm **Season**, das für ein beliebiges Datum die dazugehörige Jahreszeit bestimmt.
Die Jahreszeiten beginnen an folgenden Tagen:
21.3. . . . Frühling, 21.6. . . . Sommer, 23.9. . . . Herbst, 21.12. . . . Winter.

```
Bitte den Tag eingeben: 3
Bitte den Monat eingeben: 2
Jahreszeit am 3.2.: Winter
```

Q2-205 Römische Zahlen ⋆⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein C-Programm **RomanNumbers**, das für eine dreistellige römische Zahl den entsprechenden Wert im dezimalen Zahlensystem berechnet. Die Umrechnung erfolgt ohne Verwendung der Subtraktionsregel (z.B.: IV =

4).
Folgende Großbuchstaben sind erlaubt: I,V,X,L,C,D,M für 1, 5, 10, 50, 100, 500, 1000

```
Bitte eine roemische Zahl eingeben: XXI
Die roemische Zahl XXI hat im Dezimalsystem den Wert 21.
```

Q2-206 Hexadezimale Zahl ⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein C-Programm **HexNumberChecker**, das nach Eingabe einer zweistelligen Hexadezimalzahl feststellt, ob es sich um eine gültige Hexadezimalzahl handelt oder nicht. Anschließend erfolgt die Umrechnung in eine Dezimalzahl, sofern die Eingabe korrekt war. Es sind sowohl Klein- als auch Großbuchstaben erlaubt.

```
Bitte eine zweistellige Hexadezimalzahl eingeben: FF
Hexadezimalwert: FF  -->  Dezimalwert: 255

Bitte zweistellige Hexadezimalzahl eingeben: FX
Der eingegebene Wert FX ist keine Hexadezimalzahl.
```

Q2-3 Aufgaben zur `for`-Schleife

Q2-301 Nummernausgabe I ⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein C-Programm **NumberOutput1**, das eine ganze Zahl von der Konsole einliest und alle Zahlen von 1 bis zur eingegebenen Zahl wieder ausgibt.

```
Gib eine Zahl ein: 3
  1
  2
  3
```

Zusatz: Gib in jeder Zeile maximal fünf Zahlen aus. Jede Zahl soll auf 4 Stellen ausgegeben werden.

```
Gib eine ganze Zahl ein: 12
   1   2   3   4   5
   6   7   8   9  10
  11  12
```

Q2-302 Nummernausgabe II ⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein C-Programm **NumberOutput2**, das zwei ganze Zahlen `number1` und `number2` von der Konsole einliest, diese aufsteigend ordnet und anschließend alle Zahlen von `number1` bis `number2` auf der Konsole ausgibt. Achte auf das Ausgabeformat!

```
Erste Zahle: 7
Zweite Zahl : 2
  2 / 3 / 4 / 5 / 6 / 7
```

Zusatz: Es wird nur jede zweite Zahl ausgegeben.

```
Erste Zahle: 7
Zweite Zahl : 2
  2 / 4 / 6
```

Q2-303 Ungerade Zahlen ⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **OddNumbers**, das alle ungeraden Zahlen zwischen einem eingegebenen Start- und Endwert ausgibt. Ist der Endwert kleiner als der Startwert, so sind die beiden Werte zu tauschen. Achte auf das Ausgabeformat!

```
Startwert: 9
Endwert: 2
3, 5, 7, 9
```

Q2-304 Quadratzahlen ⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein C-Programm **SquareNumbers**, das eine Zahl von der Konsole einliest und die Quadratzahlen von 1 bis zur eingegebenen Zahl ausgibt. Achte auf das Ausgabeformat!

```
Gib eine ganze Zahl ein: 4
  1 -    1
  2 -    4
  3 -    9
  4 -   16
```

Q2-305 Summe I ⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein C-Programm **Sum1**, das eine Zahl von der Konsole einliest und die Summe aller Zahlen von 1 bis zur eingegebenen Zahl ermittelt und ausgibt.

```
Gib eine ganze Zahl ein: 5
Summe: 15
```

Q2-306 Summe II ⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein C-Programm **Sum2**, das 5 Zahlen von der Konsole einliest, diese addiert und die Summe ausgibt. Codiere die Eingabe in einer Schleife.

```
Eingabe von Zahl 1: 2
Eingabe von Zahl 2: 4
Eingabe von Zahl 3: -4
Eingabe von Zahl 4: -2
Eingabe von Zahl 5: 1
Summe: 1
```

Q2-307 Durchschnitt I ⋆⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein C-Programm **Average1**, das zwei ganze Zahlen `number1` und `number2` von der Konsole einliest und den Durchschnitt aller geraden ganzen Zahlen, die im Intervall [`number1`; `number2`] liegen, berechnet. Dazu müssen die beiden eingegebenen Zahlenwerte in aufsteigender Form vorliegen (wenn nötig, ordnen!). Ist im Intervall kein gerader Zahlenwert vorhanden, soll der Text `Keine geraden Werte vorhanden!` ausgegeben werden.

```
Eingabe von Zahl 1: 7
Eingabe von Zahl 2: 3
Berechneter Durchschnitt: 5.00


Eingabe von Zahl 1: 5
Eingabe von Zahl 2: 5
Keine geraden Werte vorhanden!
```

Q2-308 Lottozahlen I ⋆⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein C-Programm **LotteryNumbers1**, das 6 Lottozahlen zwischen 1 und 45 erzeugt und auf der Konsole ausgibt.

```
Lottozahlen 6 aus 45: 42 40 3 21 26 41
```

Q2-309 Fakultät I ⋆

Kompetenzen: Kontrollstrukturen, Algorithmen

Erstelle ein C-Programm **FacultyCalculation1**, das die Fakultät einer eingelesenen Zahl n berechnet und ausgibt. Die Fakutät ist nur für natürliche Zahlen bestimmt und ist folgendermaßen definiert:
0! = 1
$n! = n \cdot (n-1) \cdot (n-2) \cdot \cdots \cdot 1$

```
Bitte eine ganze Zahl eingeben: 5
5! = 120
```

Q2-310 Würfeln ⋆⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein C-Programm **RollTheDice**, das von der Konsole einliest, wie oft gewürfelt werden soll. Das Programm würfelt, indem entsprechend viele Zufallszahlen zwischen 1 und 6 erzeugt werden. Dabei soll mitgezählt werden, wie oft jede Augenzahl gewürfelt wurde.

```
Anzahl der Wuerfe: 1000
Augenanzahl 1: 183
Augenanzahl 2: 160
Augenanzahl 3: 168
Augenanzahl 4: 154
Augenanzahl 5: 163
Augenanzahl 6: 172
```

Q2-311 Temperaturtabelle ⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein C-Programm **TempTable**, das eine Celsius-Fahrenheit Tabelle erstellt. Für Temperaturen im Bereich von -25 Grad bis +40 Grad Celsius sollen die dazugehörigen Fahrenheitwerte berechnet und als Tabelle ausgegeben werden. Verwende für die Temperatursprünge eine Schrittweite von 5 Grad Celsius. Beachte das Ausgabeformat!
Hinweis: $C = \frac{5}{9} \cdot (F - 32)$
Ausgabe:

```
Celsius-Fahrenheit-Tabelle:
----------------------------------------
-25.0 Grad Celsius sind -13.0 Grad Fahrenheit
-20.0 Grad Celsius sind  -4.0 Grad Fahrenheit
```

```
.
.
.
+35.0 Grad Celsius sind  95.0 Grad Fahrenheit
+40.0 Grad Celsius sind 104.0 Grad Fahrenheit
```

Zusatz: Lese die untere und die obere Schranke der Grade sowie die Schrittweite von der Konsole ein.

Q2-312 Ziffernsumme I ⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **DigitSum1**, das alle zweistelligen natürlichen Zahlen ausgibt, die größer sind als das sechsfache ihrer Ziffernsumme. Führe nach zehn ausgegebenen Zahlen einen Zeilenumbruch durch.

Q2-313 Ziffernsumme II ⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **DigitSum2**, das alle dreistelligen natürlichen Zahlen ausgibt, die kleiner sind als das 30-fache ihrer Ziffernsumme. Führe nach zehn ausgegebenen Zahlen einen Zeilenumbruch durch.

Q2-314 Primzahl ⋆⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **PrimeNumber**, das prüft, ob eine vom Benutzer eingegebene Zahl eine Primzahl ist oder nicht.

```
Gib eine ganze Zahl ein: 29
Die Zahl 29 ist eine Primzahl.
```

Q2-315 Potenzieren ⋆⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **Potentiate**, das die Basis x und den Exponenten y einer Potenz x^y einliest und diese berechnet.
Das Programm soll korrekte Ergebnisse liefern, wenn der Exponent positiv, negativ oder gleich Null ist.
Die Funktion $pow()$ darf dabei NICHT verwendet werden!

Q2-316 Freier Fall ⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **FreeFall**, das eine Tabelle erstellt, in der Fallhöhe und Fallzeit aufgelistet werden. Die Ober- und Untergrenze der Fallhöhe sowie die Schrittweite werden vom Benutzer eingegeben.
Die Formel für die Fallhöhe lautet $h = \frac{1}{2} \cdot g \cdot t^2$, $g = 9.81 m/s^2$ (Erdbeschleunigung)

```
Freier Fall
-----------
Hoehenobergrenze: 100
Hoehenuntergrenze: 0
Schrittweite (in Meter): 25
```

Ausgabe:

```
Fallhoehe [m] | Fallzeit [s]
--------------|-------------
            0 |         0.00
           25 |         2.26
           50 |         3.19
           75 |         3.91
          100 |         4.52
```

Q2-317 Armstrong Zahlen ⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **ArmstrongNumbers**, das alle dreistelligen Armstrongzahlen berechnet und ausgibt.
Für dreistellige Armstrongzahlen gilt folgende Berechnungsformel: $HZE = H^3 + Z^3 + E^3$
z.B.: $153 = 1^3 + 5^3 + 3^3$

Q2-318 Turmrechnung ⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **Tower**, das eine Startzahl für eine Turmrechnung einliest und anschließend den Turm mit folgender Formatierung ausgibt:

```
Turmrechnung
------------
Startzahl: 3
        3 * 1 = 3
        3 * 2 = 6
        .   .
        .   .
        .   .
    15120 * 8 = 120960
   120960 * 9 = 1088640
  1088640 / 1 = 1088640
  1088640 / 2 = 544320
   544320 / 3 = 181440
        .   .
        .   .
        .   .
       27 / 9 = 3
```

Q2-319 Teiler finden ⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **FindDivisor**, das alle Teiler einer Zahl berechnet und ausgibt.

```
Gib eine ganze Zahl ein: 75
Die Zahl 75 ist durch folgende Zahlen teilbar: 1 3 5 15 25 75

Gib eine ganze Zahl ein: 17
Die Zahl 17 ist durch folgende Zahlen teilbar: 1 17
```

Q2-320 Quadrantensuche II ⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **Quadrants2**, das Koordinatenpunkte (x, y) mit dem Zufallsgenerator ermittelt. Dabei werden ganzzahlige Zufallswerte aus dem Intervall [-100, +100] erzeugt. Das Programm soll immer wieder neue Koordinaten ermitteln, solange der Wert von x > -80 und der Wert von y < 80 ist. Dabei sollen stets die Koordinaten ausgegebene werden. Es soll auch mitgezählt werden, in welchem Quadranten der jeweilige Punkt liegt ($x \neq 0$, $y \neq 0$). Liegt ein Punkt auf der x-Achse (x = 0) oder auf der y-Achse (y = 0), so ist dies separat zu zählen.

```
Quadrantensuche II
Koordinaten: (32, -5)
Koordinaten: (33, 0)
Koordinaten: (-46, 31)
Koordinaten: (0, 31)
Koordinaten: (-58, 5)
Koordinaten: (56, 69)
Koordinaten: (-71, -83)
Koordinaten: (14, 79)
Anzahl der Punkte im 1. Quadraten: 2
Anzahl der Punkte im 2. Quadraten: 2
Anzahl der Punkte im 3. Quadraten: 1
Anzahl der Punkte im 4. Quadraten: 1
Anzahl der Punkte auf x- oder y-Achse: 2
```

Q2-321 Quadrantensuche III ⋆⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **Quadrants3**, das zehn Koordinatenpunkte (x, y) mit dem Zufallsgenerator (float) im Bereich x ∈ [-100, +100] und y ∈ [-100, +100] ermittelt. Für jeden Koordinatenpunkt soll immer der Abstand vom Ursprung (=Länge der Linie zwischen (0,0) und dem Koordinatenpunkt) berechnet werden. Ausgegeben werden soll die größte und die kürzeste Länge. Weiters soll der Mittelwert über alle Längen berechnet und ausgegeben werden.

```
Quadrantensuche III
Laengste Linie: 131.12
Kuerzeste Linie: 39.99
Durchschnittslaenge der Linien: 74.11
```

Q2-322 Näherungsverfahren für PI ⋆⋆⋆

Kompetenzen: Kontrollstrukturen

Folgende Herren haben jeweils unterschiedliche Näherungsverfahren zur Berechnung der Kreiszahl π entwickelt:

John Wallis (1656) — das Wallissche Produkt:

$$\frac{\pi}{2} = \frac{2}{1} \cdot \frac{2}{3} \cdot \frac{4}{3} \cdot \frac{4}{5} \cdot \frac{6}{5} \cdot \frac{6}{7} \cdot \frac{8}{7} \cdot \frac{8}{9} \cdots$$

Gottfried Wilhelm Leibniz (1682) — die Leibniz-Reihe:

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \cdots$$

Leonhard Euler (1748):

$$\frac{\pi^2}{6} = \frac{1}{1^2} + \frac{1}{2^2} + \frac{1}{3^2} + \frac{1}{4^2} + \cdots$$

Schreibe ein Programm **PiCalculation**, das diese Formeln als Algorithmus umsetzt wobei die Anzahl der Reihenelemente eingelesen wird. Versuche, diese Formeln hinsichtlich Genauigkeit zu bewerten.

Die Ausgabe kann folgendermaßen aussehen:

```
Vergleich von Näherungsverfahren zur Berechnung der Kreiszahl Pi
----------------------------------------------------------------
Addierte Reihenelemente: 1000

Verfahren | Abweichung von M_PI
----------|--------------------
   Wallis |            0.001569
  Leibniz |            0.001000
    Euler |            0.000955
```

Q2-323 Bitmanipulation I ⋆⋆

Kompetenzen: Kontrollstrukturen, Operatoren

Schreibe ein Programm **Bitmanipulation1**, das eine ganze Zahl einliest und die Anzahl der gesetzten und nicht gesetzten Bits ausgibt.

Anmerkung: Negative Zahlen werden als Zweierkomplement dargestellt. Das Vorzeichen-Bit (*most significant bit*) ist in diesem Fall gesetzt.

```
Gib eine ganze Zahl (positiv/negativ) ein: -1
Anzahl der gesetzten Bits: 32
Anzahl der nicht-gesetzten Bits: 0
```

Q2-324 Bitmanipulation II ⋆⋆

Kompetenzen: Kontrollstrukturen, Operatoren

Schreibe ein Programm **Bitmanipulation2**, das eine ganze Zahl einliest. Der Benutzer kann auswählen ob er in dieser Zahl ein Bit setzen oder löschen möchte. Anschließend kann der Benutzer die Nummer des Bits eingeben. Das Programm gibt die neue Zahl und das Bitmuster vor und nach der Bitmanipulation aus.

Die Ausgabe kann folgendermaßen aussehen:

```
Gib eine ganze Zahl (positiv/negativ) ein: 32514
Binaerdarstellung:  0000 0000 0000 0000 0111 1111 0000 0010
Willst Du ein Bit (S)etzen oder (L)oeschen? S
Gib die Position ein (31 - 0): 2
Die neue Zahl lautet: 32518
Binaerdarstellung:  0000 0000 0000 0000 0111 1111 0000 0110
```

Q2-4 Aufgaben zur `while`-Schleife

Zum Üben können alle Beispiele zur for-Schleife aus Q2-3 auch mit einer while-Schleife implementiert werden!

Q2-401 Zeichen zählen ⋆

Kompetenzen: Kontrollstrukturen

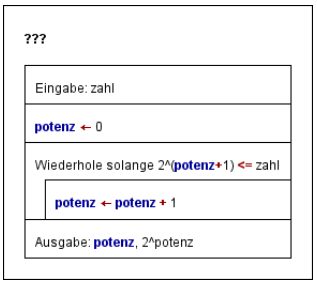
Schreibe ein Programm **CharCounter**, das beliebig viele Zeichen über die Konsole einliest und am Ende die Anzahl der eingegebenen Zeichen anzeigt.
Die Eingabe wird mit dem Raute-Zeichen beendet.

```
a
,
-
m
#
Anzahl der eingegebenen Zeichen: 4
```

Q2-402 Unbekannt I ⋆

Kompetenzen: Kontrollstrukturen, Struktogramme

Was ermittelt das im folgenden Struktogramm dargestellte Programm? Setze das Programm um.



Q2-403 Ziffernsumme III ⋆⋆

Kompetenzen: Kontrollstrukturen, Algorithmen

Schreibe ein Programm **DigitSum3**, das eine beliebig große ganze Zahl einliest und deren Ziffernsumme berechnet.

```
Bitte eine ganze Zahl eingeben: 4294967295
Ziffernsumme: 57
```

Q2-404 Zeichen analysieren ⋆⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **CharAnalysis**, das von einem zeichenweise eingegebenen Satz die Anzahl der Großbuchstaben, Kleinbuchstaben, Konsonanten, Vokale, Ziffern, Satzzeichen und Leerzeichen zählt und ausgibt. Beendet wird die Eingabe durch die Eingabe eines Punktes.

```
Satz-Eingabe:
Die Kinokarte kostet 6 Euro, das ist guenstig.
Grossbuchstaben: 3
Kleinbuchstaben: 33
Konsonanten: 20
Vokale:16
Ziffern: 1
Satzzeichen: 2
Leerzeichen: 7
```

Q2-405 Eingabestatistik ⋆⋆⋆

Kompetenzen: Kontrollstrukturen, Algorithmen

Schreibe ein Programm **InputStatistics**, das beliebigen Text zeichenweise einliest und die Anzahl aller Zeichen, aller Worte und aller Zeilen mitzählt. Die Eingabe wird mit Ctrl-Z / Strg-Z beendet. Als Trennung zwischen den Worten gelten alle Leerzeichen in beliebiger Anzahl. Leerzeichen werden in der Statistik nicht mitgezählt.

```
War einmal ein Bumerang;
War ein Weniges zu lang.
Bumerang flog ein Stück,
Aber kam nicht mehr zurück.
Publikum - noch stundenlang -
Wartete auf Bumerang.
^Z
Anzahl der eingegebenen Zeichen: 129
Anzahl der eingegebenen Worte: 26
Anzahl der Zeilen: 6
```

Q2-406 Dezimal-Binär-Konverter ⋆⋆⋆

Kompetenzen: Kontrollstrukturen, Algorithmen

Schreibe ein Programm **DezBinConverter**, das eine ganze Zahl einliest und diese vom Dezimalsystem ins Binärsystem umwandelt und ausgibt.

```
Zahl im Dezimalsystem: 35
Zahl im Binaersystem: 100011
```

Q2-407 Primfaktorenzerlegung ⋆⋆⋆

Kompetenzen: Kontrollstrukturen, Algorithmen

Schreibe ein Programm **PrimeFactorization**, das eine Zahl einliest und sie in ihre Primfaktoren zerlegt.

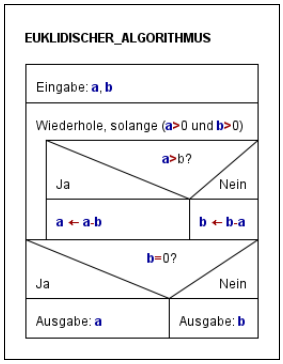
```
Beliebige ganze Zahl: 36
Primfaktoren: 2 2 3 3

Beliebige ganze Zahl: 17
Primfaktoren: 17
```

Q2-408 größter gemeinsamer Teiler ⋆⋆

Kompetenzen: Kontrollstrukturen, Algorithmen, Struktogramme

Schreibe ein Programm **HighestCommonFactor**, das zwei Zahlen einliest und den größten gemeinsamen Teiler berechnet. Implementiere den mit folgendem Struktogramm dargestellten euklidischen Algorithmus.



Zusatzaufgabe: Erweitere dein Programm und ermittle mit Hilfe des größten gemeinsamen Teilers das kleinste gemeinsame Vielfache.

Q2-409 Mittelwertberechnung I ⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **AverageCalculator1**, das 5 Gleitkommazahlen auf Variablen vom Datentyp double einliest, die Summe dieser Zahlen sowie den arithmetischen Mittelwert berechnet und ausgibt.

```
Eingabe:
======================
Zahl1: 12.3
Zahl2: -298.349
Zahl3: 424.24
Zahl4: 7.5691
Zahl5: -129.6
======================
Summe: 16.1601 Mittelwert: 3.2320
```

Q2-410 Mittelwertberechnung II ⋆⋆⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **AverageCalculator2**, das innerhalb einer Schleife immer wieder eine ganzzahlige Zufallszahl zwischen -100 und +100 erzeugt, solange der Wert der Zufallszahl größer als -50 ist. Der Mittelwert aller erzeugten Zufallszahlen ist zu berechnen und auszugeben.

```
Zahlen erstellen
Zahl: 96
Zahl: 34
Zahl: -29
Zahl: 73
Zahl: 10
Zahl: -25
Zahl: -2
Zahl: 41
Zahl: -24
-----------------------
Mittelwert: 19.33
Anzahl der Zahlen: 9
```

Q2-5 Aufgaben zur `do-while`-Schleife

Q2-501 Würfelspiel I ⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein Programm **Game1**, das immer wieder würfelt, solange die Augensumme < 100 ist. Dabei ist mitzuzählen, wie oft gewürfelt wurde. Das Würfeln wird wieder durch eine Zufallszahlenerzeugung im Intervall [1;6] umgesetzt.

```
Mit 29 Wuerfen wurde die Augensumme von 101 erreicht.
```

Q2-502 Würfelspiel II ⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **Game2**, das mitzählt, wie oft gewürfelt werden muss, bis 10-mal eine Sechs gewürfelt wurde.

```
Fuer 10 Sechser musste 67-mal gewuerfelt werden.
```

Q2-503 Fibonacci Reihe ⋆⋆

Kompetenzen: Kontrollstrukturen, Algorithmen

Schreibe ein Programm **Fibonacci**, das die Fibonacci Reihe bis zu einer eingelesenen Zahl n berechnet. Eine Zahl ergibt sich dabei immer durch die Summe der beiden Vorgänger.
$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ..., n$
Anmerkung: Die Fibonaccireihe fängt mit 0 und 1 bzw. 1 und 1 an. (Die 0 am Anfang der Reihe ist optional.)

Q2-504 Plausibilitätspruefung I ⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **Plausi1**, das die Eingabe eines ganzzahligen Wertes z wiederholt, solange z außerhalb des Intervalls $[-10, +10]$ liegt.

```
Gib einen Wert im Intervall [-10,+10] ein: 50
Gib einen Wert im Intervall [-10,+10] ein: -50
Gib einen Wert im Intervall [-10,+10] ein: -10
```

Q2-505 Plausibilitätspruefung II ⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **Plausi2**, das die Eingabe eines ganzzahligen Wertes z wiederholt, solange z innerhalb des Intervalls $[-50, +50]$ liegt.

```
Gib einen Wert ausserhalb des Intervalls [-50,+50] ein: 20
Gib einen Wert ausserhalb des Intervalls [-50,+50] ein: -50
Gib einen Wert ausserhalb des Intervalls [-50,+50] ein: 75
```

Q2-506 Summenberechnung ⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein Programm **SumCalculation**, das einen ganzzahligen positiven Summenwert einliest. Das Programm soll die Anzahl der ganzzahligen positiven Summanden beginnend mit 1 ermitteln, die addiert werden müssen bis die Summe den eingegebenen Summenwert überschritten hat.
z.B.: Summenwert 25 -> 1 + 2 + 3 + 4 + 5 + 6 + 7 = 28

```
Summenwert: 25
Anzahl der Summanden: 7
Berechnete Summe: 28
```

Zusatz: Gib die Summanden und die errechnete Summe in folgender Form aus:

```
1 + 2 + 3 + 4 + 5 + 6 + 7 = 28
```

Q2-507 zufälliger Großbuchstabe ⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **RandomLetter**, das zufällige Großbuchstaben erzeugt und sie hintereinander am Bildschirm ausgibt. Wird ein 'X', 'Y' oder 'Z' erzeugt, so wird das Programm beendet.

```
MFNKUZ
```

Q2-508 Zufallszahlen ⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **RandomNumbers**, das Zufallszahlen im Intervall $[-50 \cdots +50]$ erzeugt. Die negativen und positiven Zufallszahlen sollen getrennt summiert werden, bis der Betrag einer der beiden Summen $>= 15000$ ist. Das Programm zählt auch die Anzahl der erzeugten Zufallszahlen mit.

```
Zufallszahlen
-------------
Anzahl der positiven Zufallszahlen:   543
Summe der positiven Zufallszahlen:  14131
Anzahl der negativen Zufallszahlen:   596
Summe der negativen Zufallszahlen: -15011
```

Q2-509 Kapitalrechner ⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **CapitalCalculator**, das errechnet, wie viele Jahre ein Grundkapital verzinst werden muss, damit das resultierende Gesamtkapital mindestens das Doppelte des Grundkapitals ist.
Folgende Formel kann zur Berechnung verwendet werden:

$$Kapital = Grundkapital * (1 + (\frac{Zinssatz}{100})^n)$$

n ... Anzahl der Jahre

```
Kapitalrechner
--------------
Grundkapital: 1000.00
Zinssatz: 5.00
Nach 16 Jahren betraegt das Kapital 2078.93
```

Q2-510 Rechteckflaeche ⋆⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein Programm **RectangularArea**, das die Seiten a und b eines Rechtecks einliest. Die Seiten sind abwechselnd so lange zu halbieren, bis die resultierende Fläche $< 1/20$ der Ausgangsfläche ist.

```
Rechteckflaeche
---------------
Seitenlaenge a = 36.8
Seitenlaenge b = 19.7
Rechteckflaeche zu Beginn: 724.96
Rechteckflaeche nach Halbierungen: 22.66:
  Seitenlaenge a: 4.60 nach 3 Halbierungen.
  Seitenlaenge b: 4.93 nach 2 Halbierungen.
```

Q2-511 Quadratwurzel nach Heron ⋆⋆⋆

Kompetenzen: Kontrollstrukturen, Algorithmen

Schreibe ein Programm **SquareRootHeron**, das die Quadratwurzel aus a mit folgendem Heron-Verfahren näherungsweise berechnet:

$$x_{i+1} = \frac{1}{2}(x_i + \frac{a}{x_i})$$

Der erste Näherungswert x_1 ist a selbst.
Die Folge von Näherungswerten ist solange zu berechnen, bis sich zwei aufeinanderfolgende Werte um weniger als 0.000001 unterscheiden.

Q2-512 k-te Quadratwurzel nach Heron ⋆⋆⋆

Kompetenzen: Kontrollstrukturen, Algorithmen

Schreibe ein Programm **KthRootHeron**, das die k-te Wurzel aus a mit folgendem Heron-Verfahren näherungsweise berechnet:

$$x_{i+1} = \frac{(k-1)x_i^k + a}{kx_i^{k-1}}$$

Wobei der Radikand a als Gleitkommazahl und der Wurzelexponent k als ganze Zahl vom Benutzer einzulesen sind. Die Folge von Näherungswerten ist solange zu berechnen, bis sich zwei aufeinanderfolgende Werte um weniger als 0.000001 unterscheiden. Der erste Näherungswert x_1 ist a selbst.

Q2-513 Zahlen raten I ⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **GuessNumbers1**, das eine Zufallszahl zwischen 1 und 10 erzeugt. Der Benutzer hat maximal 3 Versuche, die Zahl zu erraten. Dabei soll bei einer falschen Zahl das Programm dem Benutzer die Hilfestellung geben, ob die zu erratende Zahl größer oder kleiner ist.
Ausgabe:
Wurde die Zahl erraten: Anzahl der Versuche
Wurde die Zahl nicht erraten: die zu erratende Zahl

```
Errate eine Zahl, die im Bereich 1 bis 10 liegt (3 Versuche):
6
Die Zahl ist groesser.
8
Die Zahl ist kleiner.
7
Gratulation. Du hast die Zahl mit 3 Versuch(en) erraten.
```

Q2-514 Zahlen raten II ⋆⋆⋆

Kompetenzen: Kontrollstrukturen, Algorithmen

Schreibe ein Programm **GuessNumbers2**, das eine Zahl erraten soll, die sich der Benutzer ausgedacht hat.
Der Benutzer überlegt sich eine Zahl zwischen 1 und 100, die der Computer erraten soll. Dabei gibt der Benutzer dem Computer eine Hilfestellung, indem er nach jeder vom Computer bestimmten Zahl 'h' für höher 'n' für niedriger und 'j' für erraten eingibt. Zum Schluss soll die erratene Zahl und die Anzahl der Versuche ausgegeben werden.

Lösungsvariante 1: Der Computer generiert jeweils eine Zufallszahl, innerhalb des durch Benutzerfeedback eingegrenzten Bereichs.

Lösungsvariante 2: Der Computer schlägt jeweils den Medianwert, des durch Benutzerfeedback eingegrenzten Bereichs vor (*divide and conquer*).

Fragen:

1. Gibt es eine Lösungsvariante die lustiger zu spielen ist als die andere? Wenn ja, warum?
2. Wie unterscheiden sich die beiden Lösungsvarianten hinsichtlich der Anzahl der Versuche? Was ist jeweils der Maximalwert bei den Versuchen?

Q2-6 Aufgaben zu verschachtelten Schleifen

Q2-601 Fakultät II ⋆⋆

Kompetenzen: Kontrollstrukturen, Algorithmen

Schreibe ein Programm **FacultyCalculation2**, das zwei positive ganze Zahlen a,b im Bereich von 1 bis 12 einliest. Berechne die Fakultät aller Zahlen die sich im Intervall [a..b] befinden und gib sie auf der Konsole aus.

```
Berechnung der Fakultaeten eines Intervalls [a,b]
-------------------------------------------------
a: 5
b: 8
5! = 120
6! = 720
7! = 5040
8! = 40320
```

Q2-602 Pythagoräisches Tripel ⋆⋆⋆

Kompetenzen: Kontrollstrukturen

Schreibe ein Programm **PythagoreanTriple**, das alle pythagoräischen Tripel bis zu einer einzugebenden Obergrenze findet.
Ein pythagoräisches Tripel wird von drei ganzen Zahlen gebildet, die Seitenlängen eines rechtwinkeligen Dreiecks sind. Dadurch gilt für sie also der pythagoräische Lehrsatz gilt. Achte darauf, dass die Elemente des Tripels (x, y, z) aufsteigend sortiert ausgegeben werden.

```
Gib eine Obergrenze für die Suche nach pythagoraeischen Tripel ein: 25
Die pythagoraeischen Tripel lauten:
(3, 4, 5)
(6, 8, 10)
(5, 12, 13)
(8, 15, 17)
(12, 16, 20)
(7, 24, 25)
```

Q2-603 Lottozahlen II ⋆⋆

Kompetenzen: Kontrollstrukturen, Algorithmen

Schreibe ein Programm **LotteryNumbers2**, das 6 **unterschiedliche** Lottozahlen zwischen 1 und 45 erzeugt und auf der Konsole ausgibt.
Zusatzaufgabe: Gib die Lottozahlen aufsteigend sortiert aus.

Q2-604 Zahlentester ⋆⋆⋆

Kompetenzen: Kontrollstrukturen

Erstelle ein C-Programm **NumberTest**, das alle dreistelligen Zahlen findet, die durch alle ihre Ziffern teilbar ist. Zum Beispiel:

```
zahl = 126 -> teilbar durch 1, teilbar durch 2, teilbar durch 6
```

Folgende Ausgabe soll das Programm erzeugen:

```
Zahl gefunden: 111
Zahl gefunden: 112
.
.
.
```

Q2-7 Aufgaben für die Weihnachtszeit

Q2-701 Weihnachtsbaumverkäufer-App ⋆⋆

Kompetenzen: Kontrollstrukturen, Algorithmen

Zeichne einen Weihnachtsbaum laut Abbildung mit dem vorgelagertem Auswahlmenü. Für das Menü kann die Zahl 1, die Zahl 2 oder die Zahl 0 eingegeben werden. Je nach Eingabe kann für die Eingabe von 1 die Höhe des Baumes oder, für die Zahl 2, die Breite des Baumes eingegeben werden können. Es sollte nur sinnvolle Werte übernommen werden. Im Anschluss wird der Baum gezeichnet und das Programm soll fragen, ob ein weiterer gezeichnet werden soll.

```
Menü:
(1) Höhe eingeben
(2) Breite eingeben
(0) Programm abbrechen
Eingabe: 1
Höhe : 6

   *
  ***
 *****
*******
   *
   *
Wollen Sie noch einen Baum zeichnen (j/n)?: n
Auf wiedersehen ...
```

Q2-702 Weihnachtsbaum ⋆

Kompetenzen: Kontrollstrukturen, Algorithmen

In den folgenden Beispielen soll die Wiederholung (for-Schleife, while) geübt werden. Folgendes ist zu beachten:

1. Jeder Stern in der Ausgabe benötigt ein printf(„*“). Es gibt also keine printf(), das mehr als einen Stern ausgibt.
2. Jedes Beispiel ist mit einer (geschachtelten) `for`-Schleife zu programmieren,
3. dann mit einer (geschachtelten) `while`-Schleife zu programmieren, und
4. `while` und `for` gekoppelt. Also im 3. Beispiel die äußere Schleife mit `for`, die innere mit `while`, das 4. Beispiel die äußere Schleife mit `while`, die innere mit `for`

Q2-702.1 10 Sterne

Gib 10 Sterne in einer Zeile aus. Beispiel:

```
**********
```

Q2-702.2 20 Sterne

Gibt 20 Sterne aus und zwar je Zeile 10 Sterne.

```
**********
**********
```

Q2-702.3 Beliebige viele Sterne

Durch Eingabe der Anzahl der Sterne in waagrechter und senkrechter Richtung, sind diese Sterne auszugeben. Hier werden die Zahlen 8 und 3 eingegeben. D.h. 8 Sterne je Zeile und insgesamt 3 Zeilen

```
Eingabe Zeilen: 3
Eingabe Sterne: 8

********
********
********
```

Q2-702.4 Dreieck I

Gib die Sterne in Dreiecksform aus: 1. Zeile ein Stern, 2. Zeile zwei Sterne, 3. Zeile drei Sterne, usw. Insgesamt sollen 5 Zeilen ausgegeben werden.

```
*
**
***
****
*****
```

Q2-702.5 Dreieck II

Kehre obiges Beispiel um. Beginne mit 5 Sterne in der 1. Zeile, 4. Sterne in der 2. Zeile, etc. Insgesamt sollen 5 Zeilen ausgegeben werden.

```
*****
****
***
**
*
```

Q2-702.6 Dreieck III

Spiegle obiges Beispiel. Beginne mit anzSpalten Sterne in der 1. Zeile, `(anzSpalten-1)`-Sterne in der 2. Zeile, etc. Insgesamt sollen `anzZeilen` Zeilen ausgegeben werden.

```
*****
 ****
  ***
   **
    *
```

Q2-702.7 Dreieck IV

Erstelle ein gleichseitiges Sternen-Dreieck, wobei die längste Seite unten ist. Verwende dazu wieder 5 Zeilen.

```
    *
   ***
  *****
 *******
*********
```

Q2-702.8 Raute I

Erstelle eine Raute wie abgebildet.

```
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
```

```
    *
```

Q2-702.9 Tannenbaum

Erstelle ein Programm, welches einen Tannenbaum aufbauend darstellt. Erstelle diesen Tannenbaum wie abgebildet.

```
      *
     ***
    *****
      *
     ***
    *****
   *******
  *********
      *
     ***
    *****
   *******
  *********
      *
     ***
    *****
   *******
  *********
 ***********
*************
      *
      *
```